

Overview

This document outlines the modular architecture for a Python application designed to compute Mixed Strategy Nash Equilibria (MSNE) for N-player games using gradient-based optimization (backpropagation). The core idea, inspired by your sketch, is to represent strategies as learnable parameters projected onto a probability simplex and iteratively update them to maximize each player's expected payoff against the (temporarily fixed) strategies of others.

1. Standard Backend Libraries

We will leverage standard, robust Python libraries:

1. **Deep Learning Framework: PyTorch** (Recommended) or TensorFlow/Keras. Essential for automatic differentiation (autograd) to compute gradients for backpropagation and for defining learnable parameters.
2. **Numerical Computation: NumPy**. For efficient creation and manipulation of payoff tensors and strategy arrays, especially when interfacing with non-PyTorch components.
3. **GUI Framework: PyQt6** (Recommended) or Tkinter. PyQt offers more features and flexibility for building the desired interface, including embedding plots.
4. **Visualization: Matplotlib** (Recommended) or Plotly. Matplotlib integrates well with PyQt and is standard for scientific plotting. Plotly offers more interactivity if desired.

2. Modular Architecture

The application will be divided into distinct modules to promote separation of concerns, testability, and maintainability:

1. **data_structures**: Defines the core objects representing the game, players, strategies, etc.
2. **game_theory_core**: Contains fundamental game theory calculations (payoffs, simplex projection).
3. **optimizer**: Implements the MSNE-finding algorithm using backpropagation.
4. **visualization**: Handles the generation of plots for strategies and losses.
5. **gui**: Manages the graphical user interface, user interaction, and integrates other modules.

3. Module Specifications

3.1. Module: data_structures

- **Purpose**: To define the fundamental data types, ensuring consistent representation throughout the application.
- **Justification**: Centralizes the definition of key entities, making the code easier to understand, test, and refactor. Prevents inconsistencies in how games or strategies are represented.
- **Key Components**:

- **Player Class:**
 - Attributes:
 - name: str - Unique identifier for the player.
 - actions: list[str] - List of action names available to the player.
 - num_actions: int - The number of actions (len(actions)).
 - strategy_params: torch.Tensor - Learnable parameters in Euclidean space (shape: (num_actions,)). Initialized randomly or uniformly. requires_grad=True is essential.
 - Invariants: name must be non-empty. actions list must not be empty (num_actions > 0). strategy_params must be a 1D tensor of size num_actions.
- **Strategy Data Class (or NamedTuple):**
 - Attributes:
 - player_name: str
 - probabilities: np.ndarray - Concrete probability distribution over actions (shape: (num_actions,)). Detached from the computation graph.
 - Invariants: probabilities.sum() must be approximately 1.0. All elements must be in [0, 1]. Length must match the corresponding player's num_actions.
- **Game Class:**
 - Attributes:
 - players: list[Player] - List of participating players.
 - payoff_tensors: dict[str, np.ndarray] - Dictionary mapping each player's name to their payoff tensor. The tensor's shape depends on the number of players and their action counts. For N players with k_1, k_2, ..., k_N actions respectively, each player's payoff tensor has shape (k_1, k_2, ..., k_N).
 - Methods: get_player_by_name(name: str) -> Player | None.
 - Invariants: Must contain at least one player. payoff_tensors must contain an entry for every player in players. The dimensions of each payoff tensor must match the action counts of the players in the correct order.
- **TrainingStatus Data Class (or NamedTuple):**
 - Attributes:
 - iteration: int - Current training step number.
 - global_loss: float | None - Overall measure of convergence/stability (optional, could be sum of player losses or max regret).
 - player_losses: dict[str, float] - Dictionary mapping player names to their individual loss values for this iteration (negative expected payoff).
 - current_strategies: dict[str, Strategy] - Snapshot of all players' strategies at this iteration.
 - Purpose: Encapsulates the state of the training at a specific point for logging and visualization.

- **Unit Tests:**
 - Verify Player initialization sets attributes correctly and strategy_params requires grad.
 - Test Strategy creation enforces invariants (sum=1, bounds).
 - Test Game initialization validates payoff tensor dimensions against player action counts. Test get_player_by_name.
 - Test TrainingStatus creation.

3.2. Module: game_theory_core

- **Purpose:** To implement core, reusable game-theoretic calculations.
- **Justification:** Isolates mathematical logic from the optimization loop and UI, making it independently testable and verifiable.
- **Key Functions:**
 - **project_to_simplex(params: torch.Tensor) -> torch.Tensor:**
 - Input: params - A 1D torch.Tensor representing raw strategy parameters.
 - Output: A 1D torch.Tensor representing the probability distribution on the simplex. Uses Softmax or an alternative projection method that ensures differentiability.
 - *Example (Softmax):* return torch.softmax(params, dim=0)
 - Type Hint: params: torch.Tensor -> torch.Tensor.
 - Conditions: Input tensor must be 1D. Output tensor elements sum to 1 and are non-negative.
 - **calculate_expected_payoffs(game: Game, strategy_profile: dict[str, torch.Tensor]) -> dict[str, torch.Tensor]:**
 - Input: game - The Game object. strategy_profile - A dictionary mapping player names to their current *projected strategy tensors* (output of project_to_simplex).
 - Output: A dictionary mapping player names to their scalar expected payoff (as a torch.Tensor of size 1 to preserve gradient information).
 - Logic: Involves tensor contractions (e.g., using torch.tensordot or torch.einsum) of the payoff tensors with the strategy tensors. For player p, their expected payoff is calculated by summing over all action profiles (a_1, ..., a_N): $\text{Sum}[\text{Payoff}_p(a_1, \dots, a_N) * \text{Prob}(a_1) * \dots * \text{Prob}(a_N)]$.
 - Type Hint: game: Game, strategy_profile: dict[str, torch.Tensor] -> dict[str, torch.Tensor].
 - Conditions: strategy_profile must contain valid probability tensors for all players in game.
- **Unit Tests:**
 - Test project_to_simplex: Verify output sums to 1, is non-negative, and handles various inputs (positive, negative, zero). Check differentiability (e.g., using torch.autograd.gradcheck).
 - Test calculate_expected_payoffs: Use simple 2x2 games (Matching Pennies, Prisoner's Dilemma, Rock-Paper-Scissors) with known pure and mixed strategies.

Verify results against manual calculations. Ensure gradients flow correctly back to the input strategy tensors.

3.3. Module: optimizer

- **Purpose:** Implements the iterative algorithm to find the MSNE using backpropagation, as described in the sketch.
- **Justification:** Encapsulates the core learning logic. This module drives the state changes in the players' strategies.
- **Key Components:**
 - **MSNEFinder Class:**
 - **Attributes:**
 - `game: Game`
 - `learning_rate: float`
 - `optimizers: dict[str, torch.optim.Optimizer]` - Dictionary mapping player names to their individual optimizers (e.g., `torch.optim.Adam` or `torch.optim.SGD`), each managing that player's `strategy_params`.
 - **Methods:**
 - **`__init__(self, game: Game, learning_rate: float, optimizer_cls=torch.optim.Adam)`:** Initializes the `game` attribute. Creates an optimizer instance in the `optimizers` dict for each player, passing their `strategy_params` to it.
 - **`step(self) -> TrainingStatus`:** Performs one full update iteration:
 1. Get current *projected* strategies for all players:
`current_projections = {p.name: project_to_simplex(p.strategy_params) for p in self.game.players}`.
 2. Initialize dictionaries for losses and gradients: `player_losses = {}`.
 3. Iterate through each player `p` to calculate their individual loss and gradients:
 - a. Create a temporary strategy profile for payoff calculation: `profile_for_p = {}`.
 - b. For each player `other_p` in the game:
 - * If `other_p` is the current player `p`, use their attached projected strategy: `profile_for_p[p.name] = current_projections[p.name]`.
 - * If `other_p` is not `p`, use their projected strategy but detach it from the computation graph: `profile_for_p[other_p.name] = current_projections[other_p.name].detach()`. This treats opponent strategies as fixed constants for `p`'s gradient calculation, as required by the sketch.
 - c. Calculate expected payoff for player `p` using the tailored profile: `payoffs = calculate_expected_payoffs(self.game, profile_for_p)`.

- d. Define p's loss as the negative expected payoff: `loss = -payoffs[p.name]`. Store it: `player_losses[p.name] = loss.item()`.
 - e. Backpropagate p's loss:
 - * Zero out gradients for p's optimizer: `self.optimizers[p.name].zero_grad()`.
 - * Compute gradients of loss w.r.t. `p.strategy_params`: `loss.backward()`. Crucially, this only computes gradients for the parameters `p.strategy_params` because other strategies were detached.
 - 4. **Update all players' parameters:** Iterate through players p and call `self.optimizers[p.name].step()`.
 - 5. Prepare and return `TrainingStatus`: Get final projected strategies (detached) using `get_current_strategies()`, calculate global loss if desired, and package into a `TrainingStatus` object.
 - **get_current_strategies(self) -> dict[str, Strategy]:** Projects each player's current `strategy_params` to the simplex and returns them as `Strategy` objects (with NumPy arrays, detached from graph).
- **Invariants:** Each optimizer in `optimizers` is linked *only* to the `strategy_params` of its corresponding player.
- **Unit Tests:**
 - Test `__init__`: Verify optimizers are created correctly for each player and linked to the right parameters.
 - Test `step()`:
 - Mock `calculate_expected_payoffs`. Verify `zero_grad()`, `backward()`, and `step()` are called for each player's optimizer.
 - Verify that `detach()` is used correctly when constructing the profile for each player's loss calculation.
 - Use gradient checking (`torch.autograd.gradcheck`) on a simplified version or with a simple game to ensure gradients are computed correctly w.r.t. only the intended player's parameters during their loss calculation.
 - Test convergence on known games (e.g., Matching Pennies -> (0.5, 0.5) for both players). Monitor `TrainingStatus`.
 - Test `get_current_strategies`: Verify output format and that probabilities sum to 1.

3.4. Module: visualization

- **Purpose:** Generates plots for display in the GUI.
- **Justification:** Decouples plotting logic from UI layout and the core optimization algorithm. Allows changing plot styles or libraries without affecting other modules.
- **Key Components:**
 - **PlotGenerator Class:**
 - Attributes: May hold references to Matplotlib figures/axes if plots are updated incrementally.
 - Methods:

- `create_simplex_plot(self, num_actions: int) -> tuple[Figure, Axes]`: Creates the initial figure and axes for a simplex plot (e.g., a triangle for 3 actions, potentially a 2D projection for >3 actions).
- `update_simplex_plot(self, ax: Axes, history: dict[str, list[np.ndarray]], player_names: list[str])`: Updates the simplex plot axes (ax) with strategy paths from history. history maps player names to lists of their strategy probability arrays over iterations. Uses different colors/markers for different players.
- `create_loss_plot(self) -> tuple[Figure, Axes]`: Creates the initial figure and axes for the loss plot.
- `update_loss_plot(self, ax: Axes, loss_history: list[TrainingStatus], player_names: list[str])`: Updates the loss plot axes (ax) showing global (if applicable) and per-player losses over iterations.
- **Helper Functions**: Potentially `project_3d_simplex_to_2d(points: np.ndarray)` or similar for visualizing simplices with >3 dimensions if needed.
- **Unit Tests:**
 - Test plot creation functions return valid Matplotlib Figure/Axes objects.
 - Test plot update functions: Provide sample history data and check if plotting commands (like `ax.plot`, `ax.set_xlabel`) are called correctly. Mock Matplotlib objects if necessary. Visual inspection of generated plots with sample data is often required. Test edge cases (empty history, single data point).

3.5. Module: gui

- **Purpose:** Provides the graphical user interface for game setup, control, and visualization.
- **Justification:** Encapsulates all UI code (PyQt), separating presentation logic from the application's core functionality.
- **Key Components (using PyQt6):**
 - **MainWindow Class** (inherits **QMainWindow**):
 - **UI Elements:**
 - Spinboxes/LineEdits for number of players.
 - Dynamically generated input fields (LineEdits) for player names and action names based on the number of players/actions.
 - A QTableWidget or similar grid structure for entering payoff values. Needs dynamic resizing and labeling based on player/action setup. Input validation is crucial here.
 - Buttons: "Setup Game", "Start Training", "Pause", "Reset".
 - Display Areas: Matplotlib canvas widgets (e.g., FigureCanvasQTAgg) embedded in the layout to display simplex and loss plots.
 - Console Output: A QTextEdit (read-only) to display status messages from the TrainingStatus objects.
 - **Internal State:**
 - `game: Game | None`

- msne_finder: MSNEFinder | None
 - plot_generator: PlotGenerator
 - strategy_history: dict[str, list[np.ndarray]]
 - loss_history: list[TrainingStatus]
 - training_thread: QThread | None
 - worker: TrainingWorker | None
 - is_training: bool
 - **Worker Thread (TrainingWorker class inheriting QObject):**
 - Runs the MSNEFinder.step() loop.
 - Has a run() method containing the loop (while self.running:).
 - Emits signals (pyqtSignal(TrainingStatus)) to pass results back to the main GUI thread safely.
 - Has methods like pause(), resume(), stop().
- **Callbacks/Slots:**
 - setup_game(): Reads player/action counts, creates input fields for names and payoffs.
 - initialize_training(): Reads all inputs (names, payoffs), validates them, creates Game, MSNEFinder, PlotGenerator, clears histories, initializes plots. Enables "Start" button.
 - start_training(): Creates and starts the TrainingWorker on a QThread. Disables setup buttons, enables "Pause".
 - pause_training(): Signals the worker to pause. Toggles button text to "Resume".
 - resume_training(): Signals the worker to resume. Toggles button text to "Pause".
 - reset_training(): Stops the worker thread, clears state, resets UI to initial configuration.
 - update_ui(status: TrainingStatus): Slot connected to the worker's signal. Appends data to histories, updates the console text, calls plot_generator.update_... methods to refresh the plots embedded in the UI.
- **Unit Tests:** (GUI testing is often integration-focused)
 - Test setup_game: Verify correct number of input fields are created.
 - Test initialize_training: Mock backend classes. Verify that a Game object can be created from sample UI inputs and that MSNEFinder is initialized. Test input validation (e.g., non-numeric payoffs, inconsistent dimensions).
 - Test basic signal/slot connections: Does clicking "Start" attempt to create and run the worker thread? (Use mocking).
 - Test update_ui: Provide a sample TrainingStatus and verify console text and plot update methods are called.

4. Integration and Workflow

1. **Launch:** User starts the application. gui.MainWindow is created and displayed.
2. **Setup:** User specifies the number of players and actions, clicks "Setup Game". UI

updates to show fields for names and the payoff table. User fills these in.

3. **Initialize:** User clicks "Initialize Training" (or maybe combined with "Start"). The gui reads all inputs, performs validation. If valid, it creates the `data_structures.Game` object, the `optimizer.MSNEFinder`, and the `visualization.PlotGenerator`. Histories are cleared, and initial plots are drawn.
4. **Start:** User clicks "Start Training". The gui creates a `TrainingWorker` with the `MSNEFinder`, moves it to a `QThread`, and starts the thread. The `TrainingWorker` enters its `run()` loop.
5. **Training Loop (Background Thread):**
 - Inside `TrainingWorker.run()`:
 - Calls `msne_finder.step()`.
 - Receives a `TrainingStatus` object.
 - Emits a signal containing the `TrainingStatus`.
 - Includes a small `time.sleep()` if needed to control update frequency or prevent excessive CPU usage. Checks for pause/stop flags.
6. **UI Update (Main GUI Thread):**
 - The `MainWindow.update_ui` slot receives the `TrainingStatus` signal.
 - It appends the strategy and loss data from the status object to the internal history lists.
 - It formats relevant info (iteration, losses) and appends it to the console `QTextEdit`.
 - It calls `plot_generator.update_simplex_plot` and `plot_generator.update_loss_plot`, passing the updated history data and the relevant Matplotlib axes embedded in the UI.
 - The Matplotlib canvas redraws the plots.
7. **Control:** User can click "Pause" to temporarily halt the background loop or "Reset" to stop it entirely and clear the state.
8. **Termination:** The loop continues until stopped or paused by the user. The application closes when the main window is closed.

This architecture provides a clear separation between the core logic, data representation, optimization algorithm, visualization, and user interface, facilitating development, testing, and future modifications.